

Vishva Rao - CS 314H Programming Assignment #1: Image Manipulation

1. Overview and Goals

The goal of this assignment was to implement a set of image transformations using two-dimensional arrays of pixels. My main goal was to make each transformation work correctly while also considering edge cases such as odd image dimensions and non-uniform arrays.

2. Solution Design

Each transformation is implemented as a subclass of `ImageEffect` with an `apply()` method that takes a 2D pixel array and returns the transformed array. I used the provided `getRed`, `getGreen`, `getBlue`, and `makePixel` methods to access and modify individual color channels.

The `NoRed`, `NoGreen`, and `NoBlue` effects set the corresponding color channel to zero, while `RedOnly`, `GreenOnly`, and `BlueOnly` preserve one channel and set the other two to zero. For `BlackAndWhite`, I calculated the average of the three RGB values and used that average for all three channels.

The reflection effects work by swapping elements. `HorizontalReflect` swaps rows from opposite ends of the image, while `VerticalReflect` swaps pixels within each row. For `Grow`, each pixel is copied into a 2×2 block, doubling the height and width. `Shrink` does the reverse by averaging each 2×2 group of pixels.

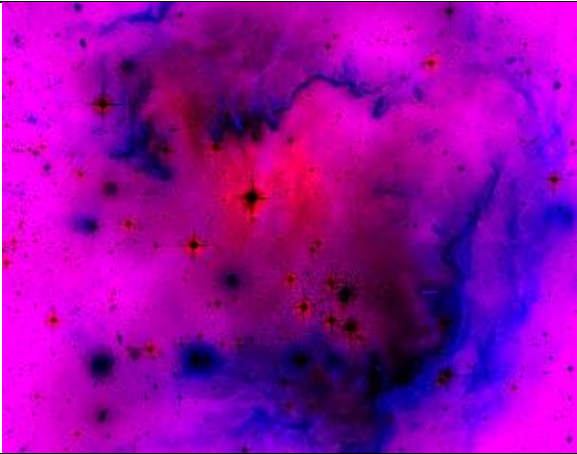
For `Shrink`, I also accounted for images with odd dimensions by removing the final row or column before performing the averaging. `VerticalReflect` handles non-uniform row lengths by padding shorter rows with black pixels. The `Threshold` effect uses a user-provided threshold and independently converts each RGB component to either 0 or 255.

3. Discussion and Evaluation

One of the more interesting parts of the assignment was handling transformations that change the dimensions of the image. `Grow` is relatively straightforward because every input pixel maps directly to four output pixels. `Shrink` is more complicated because four input pixels must be combined into a single output pixel. Averaging each RGB channel separately preserves the overall color information while reducing the image size.

Another consideration was handling images that do not have perfectly regular dimensions. `Shrink` cannot naturally average a complete 2×2 group when the image has an odd height or width. My implementation handles this by removing the final row or column before shrinking. For non-uniform arrays, `VerticalReflect` pads shorter rows with black pixels, while `Shrink` returns the original array rather than attempting to shrink it.

Creative Images

Before	After
	
	

Transformations applied:

Image 1: Shrink ---> Invert --> No green

Image 2: Grow ---> Threshold 130 ---> Reflect Vertical

For my creative images, I combined multiple transformations to create an effect that was substantially different from the original image. The before-and-after comparison shows how the

individual pixel-level operations can be combined to produce more interesting results. The assignment specifically encourages experimenting with combinations, collages, and other creative uses of the transformations.

Problems Encountered

One problem I encountered during this project was getting the code to prompt the user for a threshold value. I had to analyze the ImageEffectParam class to understand how to prompt the user for a value and read the value within the Threshold class.

Real World Application

Project one mirrors the capabilities of many photo editing apps. While it doesn't implement all features, basic color filtering and reflecting features are implemented.

4. Software Testing Methodology

I tested my transformations primarily using JUnit tests with small arrays where the expected output could be determined exactly. I used pure red, green, blue, black, white, and gray pixels to test the color transformations and verified the RGB values of the resulting pixels.

I also tested the reflection, growing, and shrinking operations using small arrays with known expected outputs. For Threshold, I specifically tested values below, equal to, and above the threshold to verify the boundary behavior.

Finally, I tested edge cases including jagged arrays and images with odd dimensions. The jagged-array test ensures that transformations do not throw exceptions when rows have different lengths, while the odd-dimension test verifies that Shrink correctly handles an extra row or column.

Overall, these tests gave me confidence that the transformations work correctly for both standard inputs and several important edge cases.